

Plano: Enriquecer a UI DESDE A ESPECIFICAÇÃO + rastreabilidade total (genérico)

Context

Dois problemas de fundo no pipeline LangNet, verificados no Uso do Solo v3 (c4871aaf) mas **genéricos** (qualquer app):

(A) A interface nasce POBRE na ESPECIFICAÇÃO — a origem. É no caso de uso (spec) que se descreve a interação real do usuário, as regras de negócio, os fluxos principais/alternativos e o **croqui (wireframe) das telas**. Mas em `app/templates/specification_prompt.py` o wireframe é um **molde hardcoded de FORMULÁRIO** (L384-397: `[Campo 1]: [____]`, "Botão Principal", "Cancelar") e o vocabulário do fluxo é form-cêntrico (L357: "campos, botões, mensagens"). O modelo **copia esse formulário para TODO UC** — não há arquétipo de mapa, gráfico, upload, galeria, timeline. A UI Spec depois só **reproduz o wireframe** (`generate_ui_spec.py` L342: "wireframe ASCII — reproduza como HTML"), e o código renderiza o JSON da tela. **Garbage in, garbage out:** a pobreza começa na spec e se propaga. Não é o modelo (qwen3.8 codifica bem) — são os **prompts** (spec → UI Spec) que só sabem formulário/tabela.

(B) Perde a rastreabilidade requisito→UC rio abaixo. Requisitos têm `FR-001..FR-026`; a spec tem a Matriz FR/UC/BR; o **ATS já carrega UC/RF Relacionado por task**. Mas o parser do `tasks.yaml` **NÃO extrai UC/RF** → o traço morre, e código/telas não referenciam requisito nenhum.

Objetivo: enriquecer a geração de UI **desde a especificação** (arquétipos de tela ricos escolhidos fielmente pelo que o UC/requisito pede — mapa, gráfico, upload, galeria, etc., de forma **genérica**, com geoespacial como um tipo entre vários), propagando pela UI Spec e pelo código; e **rastreabilidade FR/UC em todo artefato** (inclusive comentário por função e matriz consolidada). Correções nos **prompts/gerador** (produto); provar regenerando o v3 pela UI. A fonte de rastreabilidade já existe (ATS+spec) — só propagar.

Diagnóstico (para constar): onde a UI empobrece e por quê

1. **Origem** — `specification_prompt.py` : wireframe do UC é um molde de FORMULÁRIO fixo; fluxo descrito com vocabulário de form (campos/botões). Nenhum arquétipo rico. → todo UC vira formulário.
2. `generate_ui_spec.py` : catálogo de componentes CRUD-only (`text|select|table|readonly`); reproduz o wireframe pobre; sem `map/chart/image/upload/timeline` .
3. `langnetagents.py` (**code-gen**): só `_crud_screen/_report_screen/_agent_screen` — sem renderizadores ricos.

4. Riqueza do requisito (FR-002 editar polígono, FR-011 importar Shapefile, FR-013 cálculo espacial, FR-020 indicadores) **nunca é injetada** como intenção de UI em nenhum estágio. → É PROMPT+PIPELINE, não o modelo. Fix: dar vocabulário rico e **guiado pelos requisitos** nos TRÊS estágios, começando na spec.

FASE 1 — ESPECIFICAÇÃO: arquétipos de tela ricos (a origem)

`app/templates/specification_prompt.py` (seção UC + Wireframe, ~L337-427) - **Catálogo de ARQUÉTIPOS de tela/wireframe** (com exemplo ASCII de cada, não só o form): `form`, `table/list`, `map` (área do mapa + camadas + ferramenta de desenho/seleção), `chart/dashboard` (gráficos + cards de KPI), `upload/preview` (dropzone + prévia de arquivo), `gallery` (grade de imagens), `timeline/kanban`, `detail`. - **Regra de escolha do arquétipo** (fiel ao requisito): instruir o modelo a escolher pelo tipo do UC + seus **RFs Relacionados** + os dados (coluna geométrica→ `map`; UC de indicador/ relatório→ `chart`; RF/UC de "importar/anexar/upload"→ `upload/preview`; coluna de imagem→ `gallery`; itens com status→ `kanban/timeline`). - **Enriquecer o Fluxo** (L353-375): vocabulário de interação rica (ex.: "desenha a área no mapa", "seleciona a camada de zoneamento", "importa o Shapefile", "visualiza o gráfico de conformidade") — não só "preenche campo/clica botão". - Manter a regra 6.1 (consistência Fluxo↔Wireframe), agora sobre elementos ricos: todo elemento rico citado no fluxo aparece no croqui e vice-versa. - Reforçar traço: o UC já tem "RFs Relacionados" (L350) — manter obrigatório e completo.

FASE 2 — UI SPEC: catálogo de componentes rico + inferência

`prompts/generate_ui_spec.py` (`_INSTRUCTIONS` + `validate_screen`, tipos L257): - Expandir o vocabulário de `components[].type` para o **catálogo rico**: além dos de `form`, `map` (`layers`, `draw`, `srid`, `bindTo geometria`), `chart` (`chartType`), `image/gallery`, `file-upload` (`accept`), `file-preview`, `kanban`, `timeline`, `metric-card`, `rich-text`. - Design-system POR TIPO no prompt (como renderizar cada um no `mockup_html`: Leaflet, Chart.js, dropzone, etc.). `agents/langnetui.py` + `agents/langnetcoherence.py`: - `schema_columns()` retorna metadados de coluna (`geometry/imagem/arquivo`). - Derivar as **capacidades da tela** do par UC↔FR (usa o `rfs / fr` do UC) + tipos de coluna, e passar ao prompt como "capacidades exigidas" — para o LLM materializar o componente certo, coerente com o arquétipo já escolhido na spec.

FASE 3 — CODE-GEN: dispatcher de componentes + renderizadores ricos

`agents/langnetagents.py` (`_generate_business_screens`, ~L6683-6793): - Trocar o `if kind==crud/report/agent` por um **dispatcher** `_render_component(comp, ctx)` (type→JSX) — genérico/extensível. - Renderizadores ricos (React), MVP funcional cada: `map` (**Leaflet**: base OSM, camadas GeoJSON, **desenhar/selecionar** geometria→WKT SRID 4674 no campo, destaque do resultado, dispara a task); `chart` (**Recharts**); `image/gallery`; `file-upload` + `file-preview`; `kanban`; `timeline`; `metric-card`. - Deps no `package.json` do frontend gerado (leaflet/react-leaflet, recharts, react-dropzone...) conforme os tipos usados.

FASE 4 — RASTREABILIDADE FR/UC em todo artefato (espinha)

1. **tasks.yaml** — `prompts/generate_single_task_yaml.py` (`_parse_single_block`) extrai `uc / fr` do ATS; `app/routers/tasks_yaml.py` injeta determinístico `traceability: {uc, fr}` por task.
 2. **Código** — `langnetagents.py`: comentário `# Traceability: UC-004 | FR-003, FR-013` após a docstring em `_generate_deterministic_adapters` (~L3820) e nas CRUD; header nos `agents.yaml / tasks.yaml` (~L6461/6463); `data-uc / data-fr` + comentário nas telas React.
 3. **Matriz consolidada** — nova `_emit_traceability_matrix()` emite **docs/ RASTREABILIDADE.md**: FR → UC → Task → Função/arquivo → Tela → Componente(s), + seção "**FRs sem cobertura de UI/lógica**" (expõe o colapso, não esconde).
-

Arquivos-chave

- `app/templates/specification_prompt.py` — **arquétipos de tela ricos + fluxo de interação rico** (origem).
- `prompts/generate_ui_spec.py` — catálogo rico de componentes + design-system por tipo + validação.
- `agents/langnetui.py` — capacidades da tela (UC↔FR + tipos de coluna) + `fr` por tela.
- `agents/langnetcoherence.py` — metadados de coluna (geometry/imagem/arquivo).
- `agents/langnetagents.py` — dispatcher `_render_component` + renderizadores ricos; comentários de rastreabilidade; `_emit_traceability_matrix`; `package.json`.
- `prompts/generate_single_task_yaml.py` + `app/routers/tasks_yaml.py` — extrair/ injetar `traceability`.

Reuso

- Matriz FR↔UC: parse da seção 13 da spec (validado: 26 pares). ATS já tem UC/RF por task (validado).
- `render_html_to_png_b64` (`langnetui.py`) para preview PNG das telas ricas.
- PostGIS pronto (`change-dbms` + `_postgresify` desta sessão) para a tela de mapa consumir geometria 4674.
- Padrão `_annotate_tasks_*`() para anotar `traceability`.

Verificação (E2E pela UI, c4871aaf)

1. Regenerar **Especificação** (ou refinar os UCs geo) → o UC-004 (Consulta por Localização) passa a ter croqui de **mapa** (com desenhar área + camada zoneamento), UCs de dashboard com **gráfico**, UCs de importação com **upload/preview**. Conferir os wireframes ricos no doc.
2. Regenerar **UI Spec** → telas com componentes `map/chart/upload` (não form). Conferir `ui_spec_json` + mockups PNG.
3. Regenerar **tasks.yaml** → `traceability:{uc,fr}` por task.
4. Regenerar **Código** → grep: (a) `_render_component` emitiu React de mapa (Leaflet ligado a `localizacao`), gráfico, upload; (b) todo `*_deterministic` com `#`

Traceability: ; (c) docs/RASTREABILIDADE.md cobrindo FR-001..026 + lacunas; (d) data-uc/data-fr ; (e) package.json com leaflet/recharts.

5. (Opcional) subir o app PostGIS, abrir a Consulta, **desenhar a área** no mapa → regras retornam; dashboard → gráfico renderiza.

Escopo / bom senso

A raiz é a **Fase 1 (especificação)** — sem croqui rico, nada downstream fica rico; entrego os arquétipos + fluxo rico primeiro. Fases 2-3 dão o vocabulário e os renderizadores casados com os arquétipos (MVP funcional por tipo: mapa+desenho, gráfico, upload/preview primeiro; depois kanban/timeline/gallery). Fase 4 (rastreadibilidade) é determinística e transversal. Tudo genérico — geoespacial é um tipo de primeira classe entre vários — e provado regenerando o v3 pela UI.